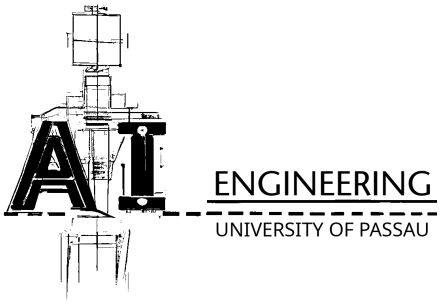
Mining Software Repositories

Exercise 1

Prof. Dr. Steffen Herbold
Winter 2026
Due on 21.10.2026



Author: Lukas Schulte

Task 1: Collecting data from Git

1.1 Exploring Git as a data source

As developers, we know Git[1] as a tool from its command line interface or from its integrations into IDEs like Visual Studio and software forges[2] like GitHub and GitLab. But as researchers, we also know Git as a software repository of information that provides data for Software Engineering Research.

Familiarize yourself with what kind of data Git can provide. Start by cloning the Apache Ignite[3] Repository. Then use the Git CLI with appropriate options to display the list of commits and the status of the files changed in each commit. Do you recognize a pattern? Which other commands / options can you use to display more information?

1.2 Mining Git with Python

For this task, we are going to utilize the PyDriller[4] library to collect data from Git. PyDriller is a Python framework that helps developers in analyzing Git repositories. With PyDriller you can easily extract information about commits, modified files, diffs, and source code. Have a look at the documentation for more information.

Hint: set the 'clone_repo_to' parameter so that you keep a local copy of the repository. Otherwise, PyDriller will spend a lot of time cloning the repository every time your script runs.

Prepare a Python development environment and work on the following tasks:

1. Create a Python script which uses PyDriller to clone the Apache Ignite repository, which is hosted on GitHub.
2. Continue using PyDriller to iterate over the commits in the repository and store information about each commit and its author.
    - Store at least the following information: date, hash, message, modified files (path), author name, author email, committer name and committer email

[1] Git Book: `https://git-scm.com/book/en/v2`
[2] Definition Forge: `https://en.wikipedia.org/wiki/Forge_(software)`
[3] Ignite: `https://github.com/apache/ignite`
[4] PyDriller `https://github.com/ishepard/pydriller`

3. Make sure you store the data in a format which you can load again later (in Task 3: Data Analysis). Consider using pandas data frames (4.3).

Task 2: Collecting data from Jira

2.1 Exploring Jira as a data source

Jira[5] is a project management tool that provides features for managing issues in a software project. We will use its REST API to collect data.

Start by familiarizing yourself with Jira. For that, have a look at the Jira instance of the Apache Foundation. You can find the Ignite project here:

`https://issues.apache.org/jira/projects/IGNITE/issues/`

2.2 Mining Jira with Python

Jira can be accessed through its REST API. The following URL can be used to retrieve the data of an issue from the Ignite Project:

`https://issues.apache.org/jira/rest/api/latest/issue/IGNITE-19111`

There are many more endpoints, for example for retrieving a list of all issues, which you can read about in the Jira API documentation[6].

Prepare a Python development environment and work on the following tasks:

1. Create a Python script which accesses the REST API of the Jira instance of the Apache Foundation for Ignite. Note that using the REST API is a requirement. Using the abstract wrapper of a third-party library to make the requests is not the point of this exercise.
2. Iterate and store all those fields of issues, which you deem to be important.
    - While iterating, make sure you are aware of pagination. Refer to the Jira REST documentation for more information.
    - While making your decision on which fields to store, also consider what you might need to answer Task 3: Data Analysis.
3. Store your aggregated data in a format which you can load later. Consider using pandas data frames (4.3).

Task 3: Data Analysis

In this task, you will perform some light data analysis. Nothing too complex, just a few graphs to see how your data looks like. Feel free to continue exploring your data beyond what is listed here.

- Load your Git data and draw a graph which visualizes the number of commits over time. Did the repository become more actively developed over time? When did it reach its peak?

[5] Jira: `https://www.atlassian.com/software/jira`
[6] Jira API: `https://docs.atlassian.com/software/jira/docs/api/REST/latest/`

- Load your Jira data and draw a similar graph. Can you make the same observation here? If not, what could be the reasons for it?
- Go back to your Git data again. Use the authors' email addresses to draw a barchart displaying the number of commits each author has added to the repository. Foolishly assuming that quantity trumps quality, who is the most active developer of your repository?

Recommended Tools

4.1 API Tools

Exploring a REST API is much easier using the right tool. If you do not already have a preferred tool for this, have a look at either Postman[7] (free version after signup) or Hoppscotch[8] (free, no signup required). Both allow you to make requests (GET, POST, PUT, etc.) to an API in a straightforward interface. And they also allow you to add authorization parameters, which may or may not be required for this exercise, depending on which project / instance of Jira you are using.

4.2 IDE

If applicable, you should use an IDE which you are already familiar with. But if you do not have a clear favorite yet, this course recommends using Visual Studio Code[9]. If you choose another IDE, we might not be able to help you with all the problems that might be related to your choice.

4.3 Pandas and Matplotlib

Pandas and Matplotlib are Python libraries that offer various tools which will enable you to store and visualize your data.

Pandas specializes in data manipulation / analysis and provides a tabular data structure called DataFrame, which is commonly used for data science tasks. One use case of a DataFrame is storing and loading data from your drive, for example as CSV files. For more information, refer to the pandas documentation.[10]

Matplotlib specializes in plotting data and can be used in Python scripts as well as Jupyter Notebooks. It can be used together with pandas to draw plots of data stored in DataFrames. For more information, refer to the Matplotlib documentation.[11]

[7] Postman: `https://www.postman.com`
[8] Hoppscotch: `https://www.hoppscotch.io`
[9] VS Code: `https://code.visualstudio.com`
[10] Pandas: `https://pandas.pydata.org/docs/`
[11] Matplotlib: `https://matplotlib.org/stable/users/index`